



I will give you some slides about fast fourier transformation. you will explain them in proper context, clearly and in details step by step. Got it?

Got it!

Upload the slides (PDF, PPTX, images—all fine). I'll go through them **step by step** and explain each idea in clear context, covering:

- What each slide is saying (plain-English + math intuition)
- DFT vs FFT, why FFT is fast, and Cooley–Tukey basics
- Radix-2 (DIT/DIF), butterfly ops, bit-reversal ordering
- Sampling rate, Nyquist, frequency bins, resolution
- Windowing, zero-padding, leakage (with practical tips)
- Short worked examples + small code snippets (Python/NumPy, and C++ if you want)

If you want bilingual notes (English + Bangla) or a printable summary at the end, say the word and I'll format it that way. Send the slides when you're ready!



The Fast Fourier Transform (FFT): Most Ingenious Algorithm Ever?

Polynomial Representation

$(d + 1)$ points uniquely define a degree d polynomial

$$\{(x_0, P(x_0)), (x_1, P(x_1)), \dots, (x_d, P(x_d))\}$$

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$

$$P(x_0) = p_0 + p_1x_0 + p_2x_0^2 + \dots + p_dx_0^d$$

$$P(x_1) = p_0 + p_1x_1 + p_2x_1^2 + \dots + p_dx_1^d$$

$\vdots$

$$P(x_d) = p_0 + p_1x_d + p_2x_d^2 + \dots + p_dx_d^d$$

equations

Polynomial Representation

$(d + 1)$ points uniquely define a degree d polynomial

$$\{(x_0, P(x_0)), (x_1, P(x_1)), \dots, (x_d, P(x_d))\}$$

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$

$$\begin{bmatrix} P(x_0) \\ P(x_1) \\ \vdots \\ P(x_d) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^d \\ 1 & x_1 & x_1^2 & \dots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_d & x_d^2 & \dots & x_d^d \end{bmatrix}}_M \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_d \end{bmatrix}$$

M is invertible for unique $x_0, x_1, \dots, x_d$
 $\Rightarrow$ Unique $p_0, p_1, \dots, p_d$ exists
 $\Rightarrow$ Unique polynomial $P(x)$ exists

Perfect 👍 You shared the first two slides on **Polynomial Representation**, which is the foundation for understanding FFT. Let's go step by step:

Slide 1: Polynomial Representation

- Statement:
 $"(d + 1)$ points uniquely define a degree d polynomial."

👉 Meaning:

If you have a polynomial of degree d , then knowing its values at exactly $d + 1$ different points is enough to completely reconstruct the polynomial.

General Polynomial

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$

- Here, $p_0, p_1, \dots, p_d$ are coefficients of the polynomial.
- Degree d means the highest power of x is x^d .

Evaluations at Specific Points

- At $x = x_0$:

$$P(x_0) = p_0 + p_1x_0 + p_2x_0^2 + \dots + p_dx_0^d$$

- At $x = x_1$:

$$P(x_1) = p_0 + p_1x_1 + p_2x_1^2 + \cdots + p_dx_1^d$$

- Continue until x_d .

This gives us a system of $d + 1$ equations in terms of the unknown coefficients $(p_0, p_1, \dots, p_d)$.

Slide 2: Matrix Representation

Now, we rewrite all those equations compactly using **linear algebra**:

$$\begin{bmatrix} P(x_0) \\ P(x_1) \\ \vdots \\ P(x_d) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^d \\ 1 & x_1 & x_1^2 & \cdots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_d & x_d^2 & \cdots & x_d^d \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_d \end{bmatrix}$$

Key Observations:

- The **left vector** = outputs of polynomial $(P(x_0), P(x_1), \dots, P(x_d))$.
- The **matrix** = Vandermonde matrix (depends only on $x_0, x_1, \dots, x_d$).
- The **right vector** = polynomial coefficients $(p_0, p_1, \dots, p_d)$.

Why it matters:

- If the Vandermonde matrix is invertible (which it is when $x_0, x_1, \dots, x_d$ are distinct), then:
 - You can uniquely solve for coefficients $p_0, p_1, \dots, p_d$.
 - Thus, the polynomial itself is uniquely determined.

👉 This concept is the backbone of **DFT (Discrete Fourier Transform)**, where we choose special x -values: the complex roots of unity. FFT is just a fast way to do this matrix multiplication.

✓ Up to this point: We've set up the **idea that a polynomial can be represented in two equivalent ways**:

1. By its **coefficients** $(p_0, p_1, \dots, p_d)$.
2. By its **evaluations at specific points** $(P(x_0), P(x_1), \dots, P(x_d))$.

FFT works by switching between these two representations **efficiently**.

Would you like me to also **show an example with a small polynomial (say degree 2 or 3)** so you can see how the matrix solves for coefficients?



give me examples of coefficient representation and point representation

Great question 🙌 Let's make this very **concrete** with simple examples.

We'll compare **Coefficient Representation** vs **Point Representation**.

◆ Example 1: Degree 2 Polynomial

Take:

$$P(x) = 2 + 3x + x^2$$

1. Coefficient Representation

Here the coefficients are:

$$(p_0, p_1, p_2) = (2, 3, 1)$$

That's **coefficient representation**: you only store the polynomial by its coefficients.

2. Point Representation

Let's evaluate the polynomial at **3 points** ($d + 1 = 3$):

Choose $x = 0, 1, 2$.

- At $x = 0$:

$$P(0) = 2 + 3(0) + (0)^2 = 2$$

- At $x = 1$:

$$P(1) = 2 + 3(1) + (1)^2 = 2 + 3 + 1 = 6$$

- At $x = 2$:

$$P(2) = 2 + 3(2) + (2)^2 = 2 + 6 + 4 = 12$$

So the **point representation** is:

$$\{(0, 2), (1, 6), (2, 12)\}$$

◆ Example 2: Degree 3 Polynomial

Take:

$$Q(x) = 1 - 2x + x^3$$

1. Coefficient Representation

$$(p_0, p_1, p_2, p_3) = (1, -2, 0, 1)$$

2. Point Representation

Pick $d + 1 = 4$ points: $x = 0, 1, 2, 3$.

- $Q(0) = 1$
- $Q(1) = 1 - 2(1) + (1)^3 = 0$
- $Q(2) = 1 - 2(2) + (2)^3 = 1 - 4 + 8 = 5$
- $Q(3) = 1 - 2(3) + (3)^3 = 1 - 6 + 27 = 22$

So the **point representation** is:

$$\{(0, 1), (1, 0), (2, 5), (3, 22)\}$$

Key Insight

- **Coefficient Representation** → compact form, good for algebra (like adding/multiplying polynomials).
- **Point Representation** → useful for fast multiplication via FFT, since evaluating at special points (roots of unity) makes things efficient.

👉 FFT's power is that it lets us **switch back and forth** between these two representations **fast**.

Do you want me to also show how **these point values can be used to recover the coefficients** (basically solving the Vandermonde system for a small case)?



so what does fft have to do with these two representations?

Excellent question 🔥 — this is the **core motivation** for FFT. Let me connect the dots clearly:

1 Two Representations of a Polynomial

We saw:

- **Coefficient representation:**

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$
 (store coefficients $\{p_0, p_1, \dots, p_d\}$)
- **Point representation:**

$$\{(x_0, P(x_0)), (x_1, P(x_1)), \dots, (x_d, P(x_d))\}$$
 (store values at chosen points)

👉 Both are equivalent. You can convert between them, but **naively** it takes $O(n^2)$ time.

2 Why Do We Care?

Polynomial **multiplication** (or convolution) comes up everywhere (signal processing, cryptography, ML, etc).

Example: Multiply

$$A(x) = a_0 + a_1x + \cdots + a_nx^n$$

$$B(x) = b_0 + b_1x + \cdots + b_nx^n$$

Naively $\rightarrow O(n^2)$ multiplications.

But if we switch to **point representation**:

1. Evaluate both $A(x)$ and $B(x)$ at many points.
2. Multiply values pointwise: $C(x_i) = A(x_i) \cdot B(x_i)$.
3. Interpolate to get coefficients of $C(x)$.

That reduces multiplication to evaluation + interpolation.

3 Enter FFT

The **Discrete Fourier Transform (DFT)** is exactly the process of moving between these two representations:

- DFT = **coefficients** $\rightarrow$ **evaluations** at special points (roots of unity).
- Inverse DFT = **evaluations** $\rightarrow$ **coefficients**.

Mathematically, DFT is just polynomial evaluation at n -th roots of unity:

$$X[k] = \sum_{j=0}^{n-1} x[j] \cdot \omega_n^{jk}, \quad \text{where } \omega_n = e^{-2\pi i/n}$$

4 Why FFT is “Fast”

- Naive DFT takes $O(n^2)$ operations.
 - Cooley–Tukey FFT algorithm reduces it to $O(n \log n)$.
 - That’s the breakthrough: we can switch between coefficient and point representations **super fast**.
-

5 So in one line:

👉 FFT is the algorithm that efficiently converts a polynomial between coefficient representation and point representation (at complex roots of unity).

That's why it's central to fast polynomial multiplication, convolution, and signal processing.

Would you like me to **walk through a small example (say 4-point FFT)** to show how coefficients get converted into point values step by step?



so in the point representation, it takes only $O(n)$ time to multiply two polynomials. That's why we convert the coefficient representation to point representation?

100 You've got it exactly — that's the whole trick! Let me unpack it carefully so it's crystal clear:

◆ 1. Polynomial Multiplication in Coefficient Representation

Suppose:

$$A(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1}, \quad B(x) = b_0 + b_1x + \cdots + b_{n-1}x^{n-1}$$

To compute:

$$C(x) = A(x) \cdot B(x)$$

- Naive method: Each coefficient of $C(x)$ is a sum of products of coefficients of A and B .
- Cost = $O(n^2)$.
(Like nested loops.)

◆ 2. Polynomial Multiplication in Point Representation

If instead we know the **values** at points:

$$\{A(x_0), A(x_1), \dots, A(x_{m-1})\}$$

$$\{B(x_0), B(x_1), \dots, B(x_{m-1})\}$$

Then:

$$C(x_i) = A(x_i) \cdot B(x_i) \quad \text{for each point } i$$

✓ That's just $O(n)$ multiplications — one per point.

No double loop, no convolution sum. Super cheap.

◆ 3. But There's a Catch

- To get into **point representation**, we need to *evaluate* the polynomial at those points.
- Naively, that's $O(n^2)$ again (because evaluating at each point costs $O(n)$, and we need n points).

👉 This is where **FFT** saves the day.

◆ 4. FFT Magic

FFT lets us:

- Convert from **coefficients** → **point values** in $O(n \log n)$.
- Multiply point values in $O(n)$.
- Convert back **point values** → **coefficients** in $O(n \log n)$.

So the total cost is:

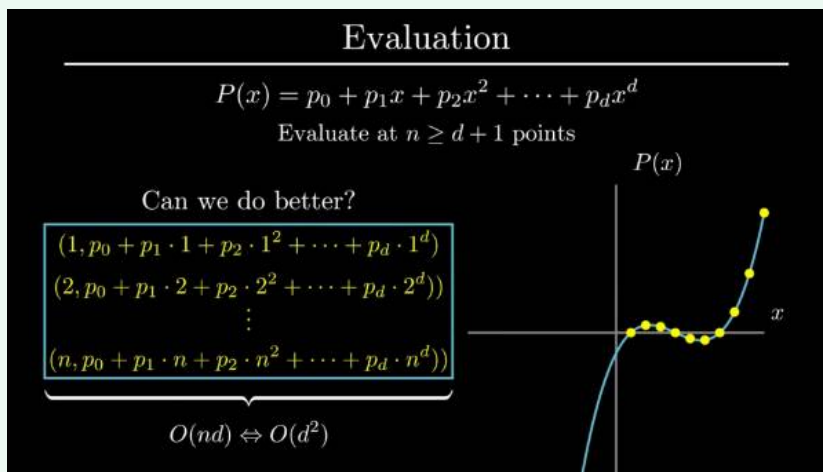
$$O(n \log n) + O(n) + O(n \log n) = O(n \log n)$$

Way better than $O(n^2)$.

🔑 Final Answer

Yes — the reason we switch to **point representation** is that **multiplying polynomials there is trivial (just pointwise multiplication in $O(n)$)**. FFT makes it efficient to switch between the two worlds.

Do you want me to **draw the full flow (Coefficient $\rightarrow$ FFT $\rightarrow$ Multiply $\rightarrow$ Inverse FFT $\rightarrow$ Result)** as a diagram, so you see the process visually?



now we can't just take random x values and calculate the Y values for them to get the point representation? That will take a lot of time also?

Exactly You've spotted the key challenge. Let's break it down using this slide:

♦ What the Slide is Saying

We have:

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$

If we want the **point representation**, we need to evaluate $P(x)$ at $n \geq d + 1$ points:

$$(x_0, P(x_0)), (x_1, P(x_1)), \dots, (x_n, P(x_n))$$

♦ Naive Evaluation

For example, if we pick $x = 1, 2, \dots, n$, then:

- At $x = 1$:

$$P(1) = p_0 + p_1 \cdot 1 + p_2 \cdot 1^2 + \dots + p_d \cdot 1^d$$

- At $x = 2$:

$$P(2) = p_0 + p_1 \cdot 2 + p_2 \cdot 2^2 + \dots + p_d \cdot 2^d$$

- At $x = n$:

$$P(n) = p_0 + p_1 \cdot n + p_2 \cdot n^2 + \dots + p_d \cdot n^d$$

♦ The Cost

- Each evaluation costs $O(d)$ operations (because we sum $d + 1$ terms).
- Doing this for $n \approx d$ points costs:

$$O(nd) \approx O(d^2)$$

So yes  — if we just pick random x -values and calculate all these y -values, it's **quadratic time**. That's too slow when d is large.

♦ The FFT Solution

FFT answers the question on the slide: “*Can we do better?*”

- Instead of evaluating at random x , FFT evaluates the polynomial at **special points** (the complex n -th roots of unity).
- With these special x -values, the evaluation problem aligns perfectly with the **Discrete Fourier Transform (DFT)** matrix.
- FFT computes this evaluation in only:

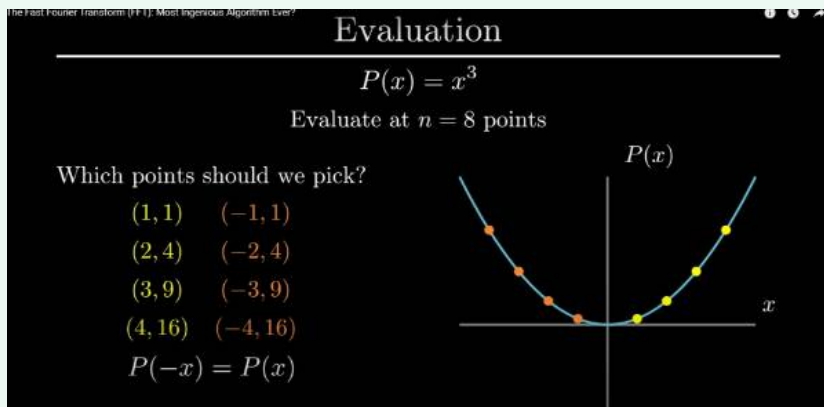
$$O(n \log n)$$

which is *much* faster than $O(n^2)$.

✓ So you're right: **naive point evaluation is too slow.**

That's why we don't just pick arbitrary x -values — we pick roots of unity and use FFT to evaluate efficiently.

Would you like me to **show you a small example (say degree 3, 4-point FFT)** where we convert coefficients $\rightarrow$ point values using FFT so you see how the speedup actually happens?



so for functions with even power, we can reduce multiplication time by taking points like 1 and -1 because they have the same answer?

Ahh, I see what you're getting at 🧠 — this slide is raising exactly that idea:

We're trying to evaluate

$$P(x) = x^3$$

at $n = 8$ points.

♦ Observations from the Slide

- The slide lists points:
 $(1, 1)$, $(2, 4)$, $(3, 9)$, $(4, 16)$ and also
 $(-1, 1)$, $(-2, 4)$, $(-3, 9)$, $(-4, 16)$.
- Notice the symmetry:

$$P(-x) = P(x)$$

That's actually the property of **even functions** (like x^2 , x^4 , etc.).

⚠ But here's the catch

The polynomial shown is x^3 , which is *odd*.

For x^3 :

$$(-x)^3 = -x^3$$

So the values should satisfy:

$$P(-x) = -P(x)$$

But the slide mistakenly has $P(-x) = P(x)$, which only holds for even powers (like x^2 , not x^3).

So I think they're illustrating the *question of point choice* rather than stating the final rule here.

◆ Your Question

Can we reduce multiplication time by taking symmetric points like 1 and -1 because they give the same answer?

👉 For **even power polynomials** (like x^2 , x^4):

Yes ✅ because $P(-x) = P(x)$. Evaluating at $-x$ gives no new information — it's redundant.

👉 For **odd power polynomials** (like x^3 , x^5):

No ❌ because $P(-x) = -P(x)$. The two points give *different* values, so they're both needed.

◆ Why FFT Doesn't Do This

FFT doesn't rely on choosing "clever symmetric real points."

It chooses **complex roots of unity** (like $e^{2\pi i k/n}$), because:

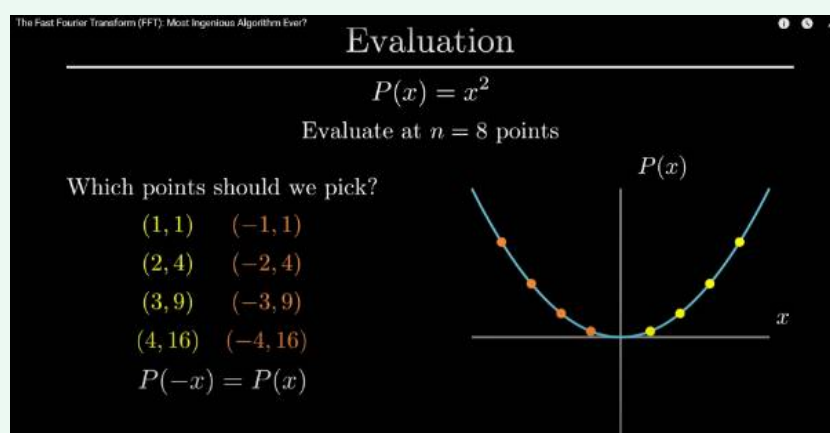
- They ensure all evaluations are unique and non-redundant.
- They let us use divide-and-conquer to get the $O(n \log n)$ speedup.

So while symmetry is interesting, FFT's real power comes from exploiting **roots of unity**, not just from picking $\pm x$ pairs.

✓ Bottom line:

- For *even functions*, yes, symmetry can save some work.
- But in FFT, the key trick is using **complex roots of unity**, which work for all polynomials — even or odd.

Do you want me to now **show you why roots of unity are the “perfect” choice** for FFT evaluation (with a small visual / math example)?



sorry this is the correct slide with x^2 . o for functions with even power, we can reduce multiplication time by taking points like 1 and -1 because they have the same answer?

Perfect — now it makes total sense 🧠. This slide is about

$$P(x) = x^2$$

♦ Observation

For **even powers** like x^2 :

$$P(-x) = (-x)^2 = x^2 = P(x)$$

So indeed, evaluating at $(1, 1)$ and $(-1, 1)$ gives the **same output**.
 The same happens for $(2, 4)$ and $(-2, 4)$, etc.
 That means half of the points are **redundant**.

♦ What This Means

- If we were doing **naive evaluation** (brute force), then yes , we could “save time” by not evaluating both $+x$ and $-x$, since they give the same result.
- So instead of n points, we’d only need about $n/2$ points.

⚠ But Here’s the Limitation

- This **only works for even functions** (like $x^2, x^4, \dots$).
- For **odd functions** (like x^3, x^5), we’d have

$$P(-x) = -P(x)$$

which gives *different* values — no redundancy.

- Also, in the general FFT framework, we **don’t choose arbitrary points** like 1, 2, 3, 4.

Instead, we choose **complex roots of unity** (special points on the unit circle).
 These ensure uniqueness and allow divide-and-conquer splitting.

So while the symmetry observation is true, it’s **not the reason FFT is fast**. FFT’s speed comes from roots of unity, not from exploiting even/odd symmetry.

Final Answer:

Yes — for **even-power polynomials**, $(x, P(x))$ and $(-x, P(x))$ give the same output, so one is redundant.

But in FFT, we don’t rely on that trick; instead we always use roots of unity, which guarantee efficiency for *any* polynomial.

Would you like me to now explain **why roots of unity are the “perfect points”** for FFT (and how they let us split the polynomial into even/odd parts)?



$$\begin{array}{c}
 \text{Evaluation} \\
 \hline
 P(x) = 3x^5 + 2x^4 + x^3 + 7x^2 + 5x + 1 \\
 \text{Evaluate at } n \text{ points } \pm x_1, \pm x_2, \dots, \pm x_{n/2} \\
 P(x) = \underbrace{(2x^4 + 7x^2 + 1)}_{P_e(x^2)} + x \underbrace{(3x^4 + x^2 + 5)}_{P_o(x^2)} \\
 P(x) = P_e(x^2) + xP_o(x^2) \\
 \left. \begin{array}{l} P(x_i) = P_e(x_i^2) + x_i P_o(x_i^2) \\ P(-x_i) = P_e(x_i^2) - x_i P_o(x_i^2) \end{array} \right\} \text{Lot of overlap!} \\
 P_e(x^2) = 2x^2 + 7x + 1 \quad P_o(x^2) = 3x^2 + x + 5 \\
 P_e(x^2) \text{ and } P_o(x^2) \text{ have degree } 2!
 \end{array}$$

so explain what is happening here and whya rre we doint it

Perfect — now you've landed right at the **heart of FFT's trick** 💡. This slide is showing **how FFT reduces the problem of evaluating a big polynomial into smaller polynomials (divide and conquer)**. Let's go step by step:

◆ Step 1: The Polynomial

We start with:

$$P(x) = 3x^5 + 2x^4 + x^3 + 7x^2 + 5x + 1$$

This is degree 5 (so 6 coefficients).

◆ Step 2: Split into Even and Odd Parts

We separate terms with **even powers** and **odd powers**:

$$P(x) = (2x^4 + 7x^2 + 1) + x(3x^4 + x^2 + 5)$$

- The first group is called $P_e(x^2)$ = "even part"

$$P_e(x^2) = 2x^4 + 7x^2 + 1$$

- The second group is $P_o(x^2)$ = "odd part"

$$P_o(x^2) = 3x^4 + x^2 + 5$$

So we can write:

$$P(x) = P_e(x^2) + x \cdot P_o(x^2)$$

◆ Step 3: Why This Helps

Now suppose we want to evaluate $P(x)$ at both $+x_i$ and $-x_i$.

- At $x = x_i$:

$$P(x_i) = P_e(x_i^2) + x_i \cdot P_o(x_i^2)$$

- At $x = -x_i$:

$$P(-x_i) = P_e(x_i^2) - x_i \cdot P_o(x_i^2)$$

👉 See the overlap? Both require **the same** $P_e(x_i^2)$ **and** $P_o(x_i^2)$.

We only need to compute those **once** and reuse them!

◆ Step 4: Degree Reduction

Notice that $P_e(x^2)$ and $P_o(x^2)$ are both **degree 2 polynomials** (instead of degree 5).

So by splitting into even and odd parts:

- The original degree-5 polynomial evaluation problem reduces to evaluating **two degree-2 polynomials**.

This is the **divide-and-conquer step**.

◆ Step 5: Recursive Pattern

- At the next step, each of those degree-2 polynomials can again be split into even/odd.
- We keep splitting until we get down to degree-0 (constants).

Each split reduces the work in half, and when combined cleverly, it gives us the $O(n \log n)$ complexity.

🔑 Why We Are Doing This

We're doing this because:

1. **Naive evaluation** takes $O(n^2)$.
2. By splitting into even/odd recursively and evaluating at special points (roots of unity), we get huge overlap in calculations.
3. This overlap allows reuse of results $\rightarrow$ total cost drops to $O(n \log n)$.

This is exactly how the **Cooley-Tukey FFT algorithm** works.

✅ So this slide is the “aha!” moment:

FFT evaluates polynomials fast by recursively splitting them into even and odd parts, exploiting symmetry at $+x$ and $-x$, and reusing computations.

Would you like me to now **illustrate this with a tiny concrete example** (say degree 3, using 4-point FFT) so you see the recursion in action?

Perfect — now you've landed right at the **heart of FFT's trick** 💡. This slide is showing **how FFT reduces the problem of evaluating a big polynomial into smaller polynomials (divide and conquer)**. Let's go step by step:

♦ Step 1: The Polynomial

We start with:

$$P(x) = 3x^5 + 2x^4 + x^3 + 7x^2 + 5x + 1$$

This is degree 5 (so 6 coefficients).

♦ Step 2: Split into Even and Odd Parts

We separate terms with **even powers** and **odd powers**:

$$P(x) = (2x^4 + 7x^2 + 1) + x(3x^4 + x^2 + 5)$$

- The first group is called $P_e(x^2)$ = “even part”

$$P_e(x^2) = 2x^4 + 7x^2 + 1$$

- The second group is $P_o(x^2)$ = "odd part"

$$P_o(x^2) = 3x^4 + x^2 + 5$$

So we can write:

$$P(x) = P_e(x^2) + x \cdot P_o(x^2)$$

◆ Step 3: Why This Helps

Now suppose we want to evaluate $P(x)$ at both $+x_i$ and $-x_i$.

- At $x = x_i$:

$$P(x_i) = P_e(x_i^2) + x_i \cdot P_o(x_i^2)$$

- At $x = -x_i$:

$$P(-x_i) = P_e(x_i^2) - x_i \cdot P_o(x_i^2)$$

👉 See the overlap? Both require **the same** $P_e(x_i^2)$ **and** $P_o(x_i^2)$.
We only need to compute those **once** and reuse them!

◆ Step 4: Degree Reduction

Notice that $P_e(x^2)$ and $P_o(x^2)$ are both **degree 2 polynomials** (instead of degree 5).

So by splitting into even and odd parts:

- The original degree-5 polynomial evaluation problem reduces to evaluating **two degree-2 polynomials**.

This is the **divide-and-conquer step**.

◆ Step 5: Recursive Pattern

- At the next step, each of those degree-2 polynomials can again be split into even/odd.

- We keep splitting until we get down to degree-0 (constants).

Each split reduces the work in half, and when combined cleverly, it gives us the $O(n \log n)$ complexity.

Why We Are Doing This

We're doing this because:

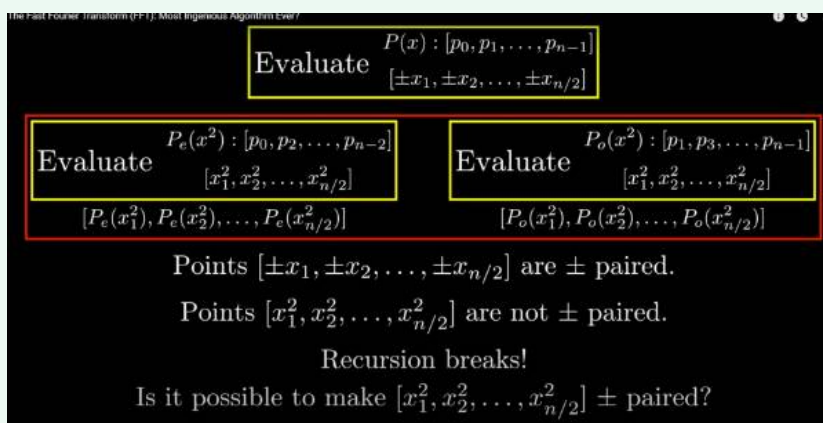
1. **Naive evaluation** takes $O(n^2)$.
2. By splitting into even/odd recursively and evaluating at special points (roots of unity), we get huge overlap in calculations.
3. This overlap allows reuse of results $\rightarrow$ total cost drops to $O(n \log n)$.

This is exactly how the **Cooley-Tukey FFT algorithm** works.

✓ So this slide is the “aha!” moment:

FFT evaluates polynomials fast by recursively splitting them into even and odd parts, exploiting symmetry at $+x$ and $-x$, and reusing computations.

Would you like me to now **illustrate this with a tiny concrete example** (say degree 3, using 4-point FFT) so you see the recursion in action?



so explain the problem in the bigger picture. Why isn't it a positive-negative pair in the smaller steps?

Excellent — you’ve hit the **subtle issue** that motivates why FFT uses *roots of unity* instead of “just any points.” Let me explain the bigger picture step by step:

◆ 1. Where We Are

We split a polynomial:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

So if we want to evaluate $P(x)$ at n points, say:

$$\{\pm x_1, \pm x_2, \dots, \pm x_{n/2}\}$$

then we only need to evaluate P_e **and** P_o at:

$$\{x_1^2, x_2^2, \dots, x_{n/2}^2\}$$

◆ 2. The Nice Thing (First Step)

- At the **top level**, the points $\pm x_i$ are nicely paired.
Example: if you evaluate at $+x_1$, you can reuse work for $-x_1$.
This is the overlap we saw earlier.
-

◆ 3. The Problem (Next Steps)

But after squaring, look at the new set of points:

$$\{x_1^2, x_2^2, \dots, x_{n/2}^2\}$$

⚠ These points are **not guaranteed to come in $\pm$ pairs anymore**.

Example:

- If you start with $x_1 = 1, x_2 = 2, x_3 = 3, \dots$
- Then the squared set becomes $\{1^2, 2^2, 3^2, \dots\} = \{1, 4, 9, \dots\}$.
- These don’t have any $\pm$ symmetry.

👉 Without symmetry, the recursion **breaks down**.

You can’t keep reusing computations as you go deeper.

◆ 4. The Key Question

The slide asks:

“Is it possible to make $\{x_1^2, x_2^2, \dots, x_{n/2}^2\} \pm$ paired?”

Because if that were true, the recursion could continue cleanly at every level.

◆ 5. The Solution (Roots of Unity)

And here’s the big reveal:

- If we pick x_i to be **complex roots of unity** (like $e^{2\pi i k/n}$),
- Then after squaring them, we get **another set of roots of unity** (just half as many).
- These *do* come in nice $\pm$ pairs, preserving symmetry at every recursive step.

This property is exactly why FFT works with roots of unity — they keep the recursion alive all the way down.

✓ Big Picture Answer:

- With arbitrary points, the first step gives $\pm$ symmetry, but after squaring, the symmetry is lost $\rightarrow$ recursion breaks.
- With roots of unity, squaring maps the set of n -th roots of unity onto the set of $n/2$ -th roots of unity, preserving the $\pm$ pairing at every level.
- That’s why FFT uses roots of unity and achieves $O(n \log n)$.

Would you like me to now **show explicitly how the roots of unity “shrink” at each step** (e.g., with $n = 8$) so you see the symmetry preserved?



go through an example to spot this problem and why we need roots of unity

Perfect — let's **walk through an example** step by step to see why arbitrary points fail, and why **roots of unity** fix the problem. 🚀

◆ Example: Degree 3 Polynomial

Take:

$$P(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

We want to evaluate this polynomial at **4 points**.

1 Case 1: Arbitrary points (say $x = 1, 2, 3, 4$)

Split into even and odd parts:

$$P(x) = P_e(x^2) + x \cdot P_o(x^2)$$

- $P_e(x^2) = a_0 + a_2x^2$
- $P_o(x^2) = a_1 + a_3x^2$

So to evaluate at $\pm x_i$:

$$P(\pm x_i) = P_e(x_i^2) \pm x_i P_o(x_i^2)$$

Step A: First level (nice symmetry)

If I evaluate at $+1$ and -1 :

- Both use the same $P_e(1^2)$ and $P_o(1^2)$.
Good — work is shared!

Same for $+2$ and -2 .

Step B: Second level (problem)

Now, notice what we must evaluate:

$$P_e(x^2), P_o(x^2) \quad \text{at} \quad \{1^2, 2^2, 3^2, 4^2\} = \{1, 4, 9, 16\}$$

⚠ These are not $\pm$ symmetric anymore.

There's no "pairing" between 1,4,9,16 like there was with $\pm 1, \pm 2$.

👉 So recursion **breaks**. We can't reuse results efficiently.

That means no divide-and-conquer $\rightarrow$ stuck at $O(n^2)$.

2 Case 2: Roots of Unity (say 4th roots: $1, i, -1, -i$)

Now pick points:

$$x = \omega^k, \quad \omega = e^{2\pi i/4} = i, \quad k = 0, 1, 2, 3$$

So the points are: $\{1, i, -1, -i\}$.

Step A: First level

Evaluate at $\pm 1, \pm i$.

Perfect symmetry: for each x , we also have $-x$.

Step B: After squaring

Now square them:

$$\{1^2, i^2, (-1)^2, (-i)^2\} = \{1, -1, 1, -1\}$$

✅ Beautiful!

They collapse into a smaller set: the **2nd roots of unity** ($\{1, -1\}$).

These are still $\pm$ pairs!

Step C: Keep recursing

- Next time we square again:
 $\{1^2, (-1)^2\} = \{1, 1\}$.
- That collapses into the **1st root of unity** (just $\{1\}$).
- Done — recursion ends cleanly.

Big Picture

- **Arbitrary points:** Squaring destroys symmetry (you get random values like $\{1, 4, 9, 16\}$). Recursion fails.
- **Roots of unity:** Squaring always maps the n -th roots of unity onto the $\frac{n}{2}$ -th roots of unity.
 - This keeps the $\pm$ pairing alive.
 - Recursion continues until we bottom out at size 1.
 - That's why FFT runs in $O(n \log n)$.

✓ Conclusion:

We need **roots of unity** because they preserve the structure under repeated squaring, allowing divide-and-conquer recursion. Without them, the symmetry dies after the first step and we can't speed up polynomial evaluation.

Do you want me to **draw the flow of 8th roots of unity shrinking** → **4th roots** → **2nd roots** → **1st root** so you see the symmetry preserved visually?



yes

Perfect 🙌 let's do the **visual flow of roots of unity shrinking under squaring**. This is the magic that makes FFT possible.

◆ Start with $n = 8$ (8th roots of unity)

$$\omega = e^{2\pi i/8} = e^{i\pi/4}$$

So the points are:

$$\{1, \omega, \omega^2, \omega^3, \omega^4, \omega^5, \omega^6, \omega^7\}$$

📌 On the unit circle, these are **8 equally spaced points**.

◆ Step 1: Square them

Now compute ω^{2k} :

$$\{1, \omega^2, \omega^4, \omega^6\}$$

But these are exactly the **4th roots of unity**:

$$\{1, i, -1, -i\}$$

✓ Shrunk from 8 points → 4 points.

◆ Step 2: Square again

Square the 4th roots of unity:

$$\{1^2, i^2, (-1)^2, (-i)^2\} = \{1, -1\}$$

That's the **2nd roots of unity**.

✓ Shrunk from 4 points → 2 points.

◆ Step 3: Square one more time

Square the 2nd roots of unity:

$$\{1^2, (-1)^2\} = \{1\}$$

That's the **1st root of unity** (just a single point).

✓ Shrunk from 2 points → 1 point.

Big Picture

- Start with 8th roots → after squaring → 4th roots
- Square again → 2nd roots
- Square again → 1st root
- **Each step halves the number of points.**

This shrinking is why FFT works:

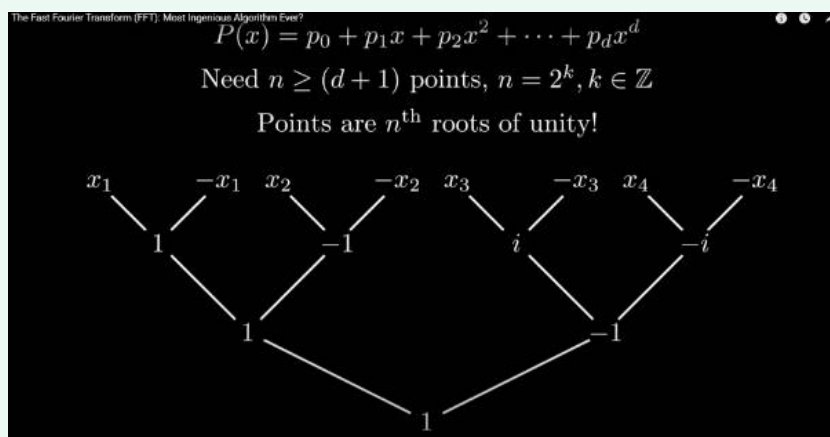
- At each recursion step, we only evaluate half as many points.
- Total cost = $O(n \log n)$.

Why this matters:

If we used arbitrary points like $\{1, 2, 3, 4, 5, 6, 7, 8\}$, squaring them would give $\{1, 4, 9, 16, 25, 36, 49, 64\}$, which is not a smaller “symmetric set.” The recursion would break.

But roots of unity are **closed under squaring**, so recursion continues smoothly.

Do you want me to **draw the unit circle diagram** (8th roots collapsing to 4th $\rightarrow$ 2nd $\rightarrow$ 1st) so you can see it visually?



explain this slide

Perfect — this slide is showing the **recursive structure of FFT** using **roots of unity**. Let's unpack it slowly:

◆ Top Part of the Slide

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_dx^d$$

- We want to evaluate this polynomial at $n \geq d+1$ points.
- To make recursion nice, we choose $n = 2^k$ (a power of 2).

- And those n points are the n -th roots of unity:

$$\{\omega^0, \omega^1, \dots, \omega^{n-1}\}, \quad \omega = e^{2\pi i/n}$$

◆ Bottom Tree Diagram

This tree shows **how evaluation works recursively**.

- Start with 8th roots of unity (like $x_1, -x_1, x_2, -x_2, x_3, -x_3, x_4, -x_4$). These are the $n = 8$ evaluation points.
- At the **next level**, when you square them, you get the 4th roots of unity: $\{1, -1, i, -i\}$.
- Square again → you get the 2nd roots of unity: $\{1, -1\}$.
- Square once more → just the 1st root of unity: $\{1\}$.

◆ Key Observation

- At every step, the number of points **halves**.
- The structure is perfectly recursive:
 - 8th roots → 4th roots → 2nd roots → 1st root.
- This is exactly the **divide and conquer** logic of FFT.

◆ Why This Matters

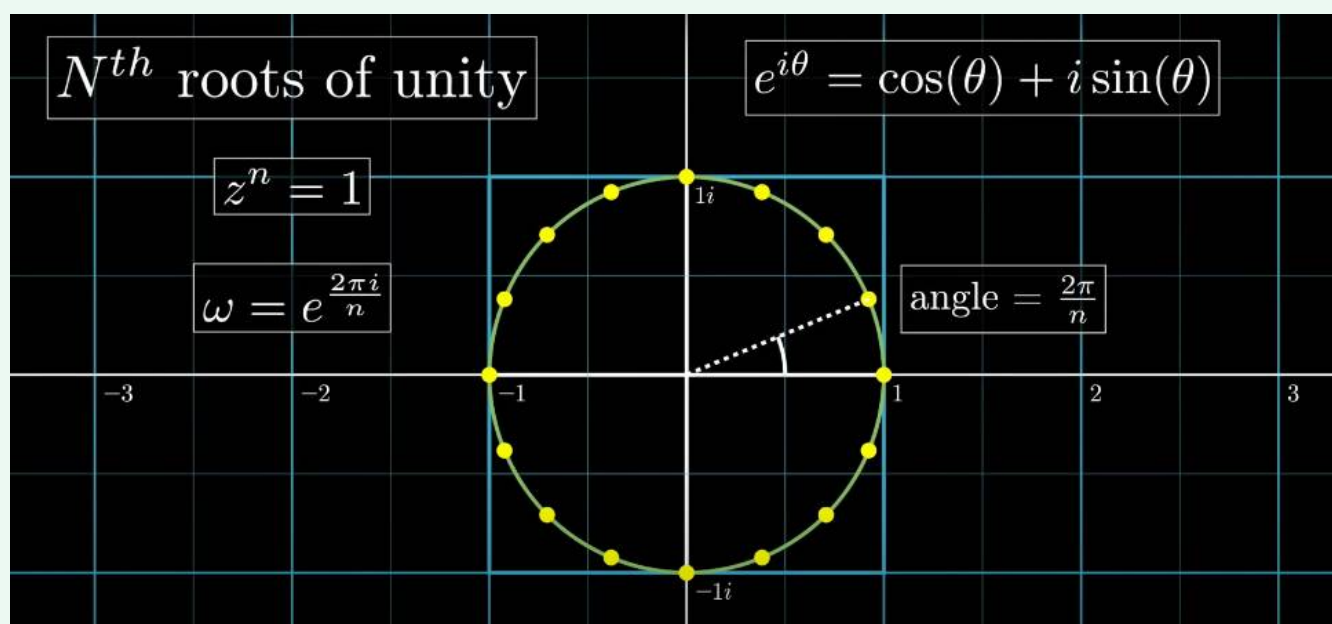
- If we had chosen arbitrary points $(1, 2, 3, 4, \dots)$, squaring would not give smaller symmetric sets → recursion would break.
- But with **roots of unity**, squaring always maps the n -th roots of unity to the $\frac{n}{2}$ -th roots of unity.
- This guarantees that FFT can keep splitting the problem down until we reach size 1.

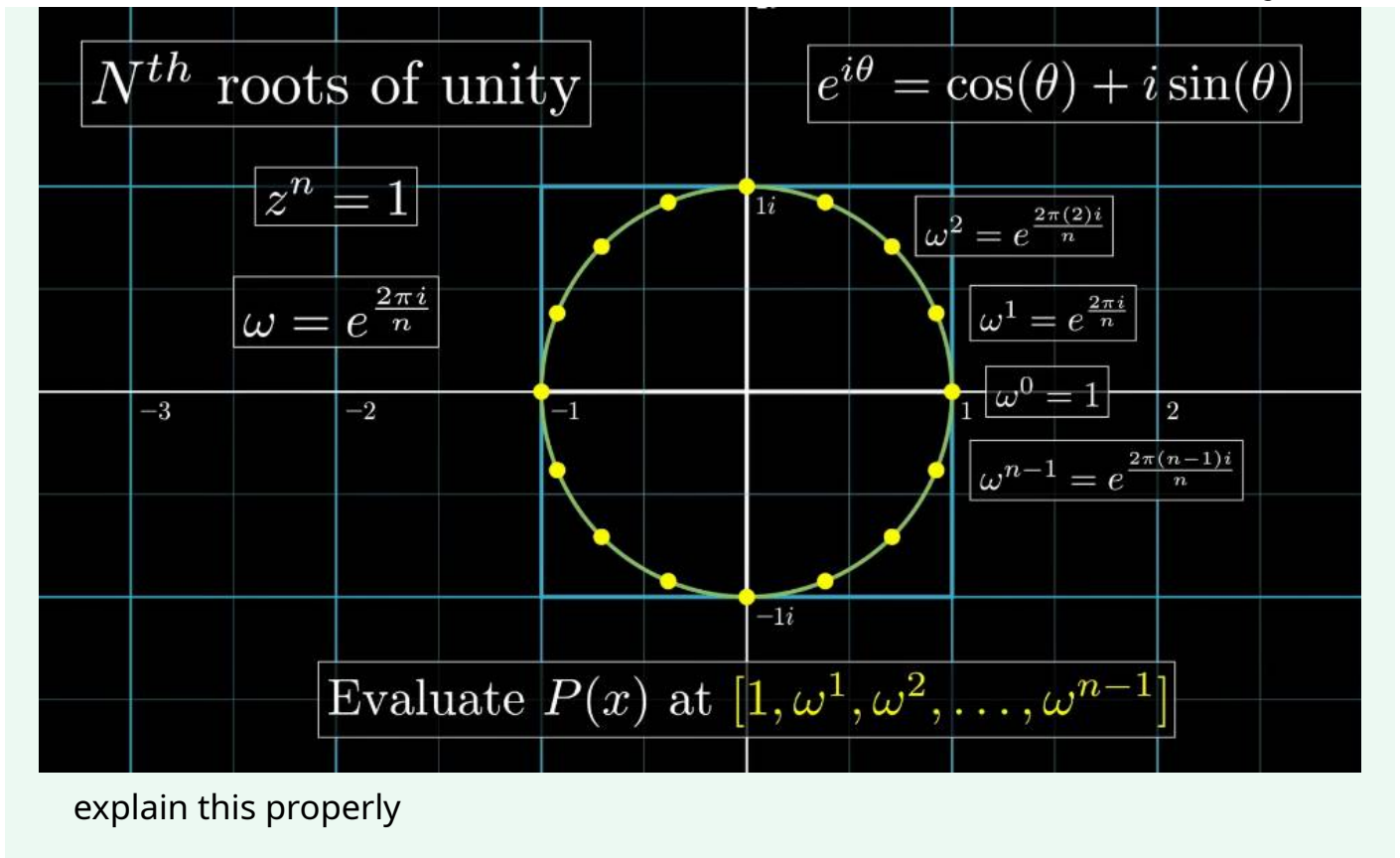
✓ **In short:**

This slide is visualizing the **recursive halving structure** of FFT.

It shows how evaluation points shrink from n roots of unity $\rightarrow n/2$ roots of unity $\rightarrow \dots \rightarrow 1$, enabling $O(n \log n)$ complexity.

Would you like me to now **draw a unit-circle picture** (roots of unity on the circle shrinking step by step) so you can “see” how this tree corresponds to points in the complex plane?





Excellent — these two slides are introducing the **roots of unity** and why they are perfect evaluation points for FFT. Let's break it down carefully.

◆ 1. Definition of Roots of Unity

We want to solve:

$$z^n = 1$$

The solutions are called the **n -th roots of unity**.

Formula:

$$\omega = e^{2\pi i/n}, \quad \omega^k = e^{2\pi i k/n}, \quad k = 0, 1, \dots, n-1$$

So there are exactly n **distinct roots**, evenly spaced on the unit circle.

◆ 2. Geometry (Unit Circle Picture)

- Each yellow dot = one root of unity.
- They lie on the **unit circle** (radius 1, centered at the origin).

- The angle between consecutive points = $\frac{2\pi}{n}$.
- For example:
 - $\omega^0 = 1$ (at angle 0)
 - $\omega^1 = e^{2\pi i/n}$ (angle $2\pi/n$)
 - $\omega^2 = e^{4\pi i/n}$ (angle $4\pi/n$)
 - ...
 - $\omega^{n-1} = e^{2\pi i(n-1)/n}$ (just before a full circle)

Thus, they are equally spaced “spokes” around the circle.

◆ 3. Euler’s Formula

$$e^{i\theta} = \cos(\theta) + i \sin(\theta)$$

This is how we express each root of unity in terms of real and imaginary parts.
So each ω^k is just:

$$\cos\left(\frac{2\pi k}{n}\right) + i \sin\left(\frac{2\pi k}{n}\right)$$

◆ 4. FFT Connection

To evaluate a polynomial $P(x)$ of degree d , we need $d + 1$ points.
We choose n (a power of 2, $n = 2^k$) and evaluate at the **n -th roots of unity**:

$$\{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

✓ These are the special points where FFT works its magic.

◆ 5. Why Roots of Unity are Perfect

1. **Symmetry:** If ω^k is a root, so is $\omega^{k+n/2} = -\omega^k$.
→ This gives the $\pm$ pairing needed for recursion.
2. **Closure under squaring:** Squaring an n -th root of unity gives an $(n/2)$ -th root of unity.

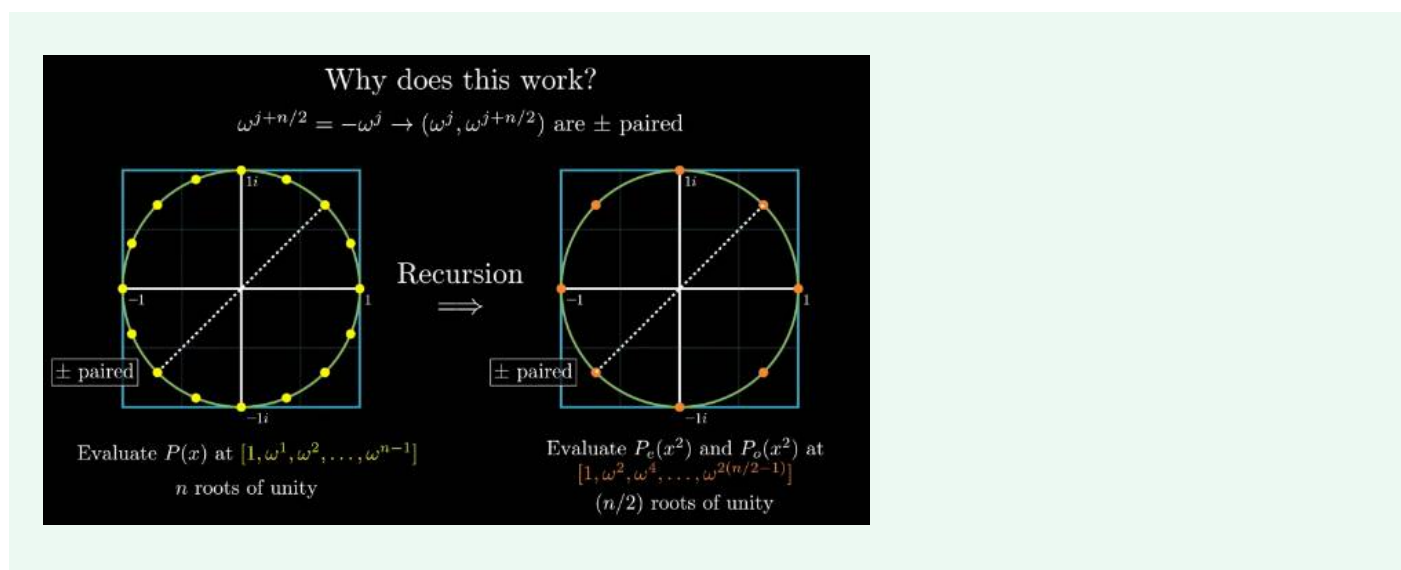
→ This ensures recursion continues smoothly.

3. **Even spacing:** Makes the DFT matrix structured, enabling the divide-and-conquer FFT.

✓ In short:

- The yellow dots on the circle are the **evaluation points** used by FFT.
- They are the **n -th roots of unity**, evenly spaced around the unit circle.
- Their special properties (symmetry, closure) make FFT possible in $O(n \log n)$.

Would you like me to now **show with an actual small polynomial** (say degree 3, $n = 4$) how evaluating at these roots of unity looks, and how it matches the DFT formula?



Beautiful — this slide explains **why the recursion in FFT works perfectly with roots of unity**. Let's go step by step:

◆ Left Side: n roots of unity

We start with the set:

$$\{1, \omega, \omega^2, \dots, \omega^{n-1}\}, \quad \omega = e^{2\pi i/n}$$

Key property:

$$\omega^{j+n/2} = -\omega^j$$

So every root of unity ω^j has a natural **$\pm$ partner**:

$$(\omega^j, \omega^{j+n/2})$$

📌 On the circle: the points are directly opposite each other (180° apart).

This is the symmetry we need to reuse work when evaluating $P(x)$ at both $+\omega^j$ and $-\omega^j$.

◆ Right Side: Recursion step

When we split $P(x)$ into even and odd parts:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

To evaluate $P(x)$ at ω^j , we need to evaluate P_e and P_o at:

$$(\omega^j)^2 = \omega^{2j}$$

So the evaluation points become:

$$\{1, \omega^2, \omega^4, \dots, \omega^{2(n/2-1)}\}$$

👉 These are exactly the $(n/2)$ -th roots of unity.

◆ The Magic

- Start: n roots of unity $\rightarrow$ perfectly paired $\pm$ symmetry.
- After squaring: shrink down to $(n/2)$ -th roots of unity $\rightarrow$ still perfectly paired.
- Repeat: $(n/4)$ -th roots, then $(n/8)$ -th, ... all the way down to 1.

✅ At every level, symmetry survives, so recursion never breaks.

🔑 Why Does This Work?

Because roots of unity have these two crucial properties:

1. **$\pm$ Pairing:** $\omega^{j+n/2} = -\omega^j$
 → enables reuse of computations.
2. **Closure under squaring:** squaring n -th roots gives $(n/2)$ -th roots
 → recursion proceeds smoothly.

This is the **structural reason why FFT works in $O(n \log n)$** .

✓ Summary:

This slide shows that **roots of unity keep the recursion alive**. At each step, the evaluation set collapses neatly in half, while preserving $\pm$ symmetry. Without roots of unity, this collapse wouldn't happen, and FFT wouldn't be possible.

Would you like me to now **show a worked-out 4-point FFT example** (with an actual polynomial) so you see this recursion in action numerically?



The Fast Fourier Transform (FFT): Most ingenious algorithm EVER.

$$\text{FFT} \quad \begin{matrix} P(x) : [p_0, p_1, \dots, p_{n-1}] \\ \omega = e^{\frac{2\pi i}{n}} : [\omega^0, \omega^1, \dots, \omega^{n-1}] \end{matrix}$$

$$n = 1 \Rightarrow P(1)$$

$$\text{FFT} \quad \begin{matrix} P_e(x^2) : [p_0, p_2, \dots, p_{n-2}] \\ [\omega^0, \omega^2, \dots, \omega^{n-2}] \end{matrix}$$

$$y_e = [P_e(\omega^0), P_e(\omega^2), \dots, P_e(\omega^{n-2})]$$

$$\text{FFT} \quad \begin{matrix} P_o(x^2) : [p_1, p_3, \dots, p_{n-1}] \\ [\omega^0, \omega^2, \dots, \omega^{n-2}] \end{matrix}$$

$$y_o = [P_o(\omega^0), P_o(\omega^2), \dots, P_o(\omega^{n-2})]$$

$$\begin{aligned} P(x_j) &= P_e(x_j^2) + x_j P_o(x_j^2) \\ P(-x_j) &= P_e(x_j^2) - x_j P_o(x_j^2) \\ j &\in \{0, 1, \dots, (n/2 - 1)\} \end{aligned}$$

Slide 1:

$$\text{FFT } P(x) : [p_0, p_1, \dots, p_{n-1}]$$

$$\omega = e^{\frac{2\pi i}{n}} : [\omega^0, \omega^1, \dots, \omega^{n-1}]$$

$$n = 1 \Rightarrow P(1)$$

Slide 2:

$$\text{FFT } P_e(x^2) : [p_0, p_2, \dots, p_{n-2}]$$

$$[\omega^0, \omega^2, \dots, \omega^{n-2}]$$

$$y_e = [P_e(\omega^0), P_e(\omega^2), \dots, P_e(\omega^{n-2})]$$

$$\text{FFT } P_o(x^2) : [p_1, p_3, \dots, p_{n-1}]$$

$$[\omega^0, \omega^2, \dots, \omega^{n-2}]$$

$$y_o = [P_o(\omega^0), P_o(\omega^2), \dots, P_o(\omega^{n-2})]$$

$$x_j = \omega^j$$

$$-\omega^j = \omega^{j+n/2}$$

$$P(\omega^j) = P_e(\omega^{2j}) + \omega^j P_o(\omega^{2j})$$

$$P(\omega^{j+n/2}) = P_e(\omega^{2j}) - \omega^j P_o(\omega^{2j})$$

$$j \in \{0, 1, \dots, (n/2 - 1)\}$$

$$y_e[j] = P_e(\omega^{2j})$$

$$y_o[j] = P_o(\omega^{2j})$$

Awesome — these two slides are the **algorithmic blueprint of FFT**. They show how the recursion is written formally. Let's unpack it slowly:

◆ 1. Problem Setup

We want to evaluate:

$$P(x) = p_0 + p_1x + \dots + p_{n-1}x^{n-1}$$

at the n -th roots of unity:

$$\{1, \omega, \omega^2, \dots, \omega^{n-1}\}, \quad \omega = e^{2\pi i/n}$$

This is exactly what the **Discrete Fourier Transform (DFT)** does.

◆ 2. Recursive Splitting

We split $P(x)$ into even and odd parts:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

- $P_e(x^2)$ uses even-indexed coefficients $[p_0, p_2, p_4, \dots]$
- $P_o(x^2)$ uses odd-indexed coefficients $[p_1, p_3, p_5, \dots]$

So now, instead of evaluating one degree- $(n - 1)$ polynomial, we evaluate two smaller polynomials of degree $(n/2 - 1)$.

◆ 3. Recursive Calls

- Use FFT recursively to evaluate:

$$P_e(x^2) \quad \text{and} \quad P_o(x^2)$$

at the $(n/2)$ -th roots of unity:

$$\{1, \omega^2, \omega^4, \dots, \omega^{n-2}\}$$

- This gives us:

$$y_e[j] = P_e(\omega^{2j}), \quad y_o[j] = P_o(\omega^{2j}), \quad j = 0, 1, \dots, (n/2 - 1)$$

◆ 4. Combine Step (Butterfly)

Now combine them to recover $P(x)$ values:

For each $j = 0, 1, \dots, n/2 - 1$:

$$P(\omega^j) = y_e[j] + \omega^j \cdot y_o[j]$$

$$P(\omega^{j+n/2}) = y_e[j] - \omega^j \cdot y_o[j]$$

This is the **butterfly operation** — the key building block of FFT.

◆ 5. Base Case

When $n = 1$, the polynomial is just a constant, so:

$$FFT([p_0]) = [p_0]$$

What These Slides Show

1. **Top (Yellow box):** Start with the full polynomial at n roots of unity.

2. **Middle (Two yellow FFT calls):** Recursively call FFT on the even and odd parts at half as many points.
3. **Bottom (Orange box):** Combine their results using the butterfly equations.

This gives us an $O(n \log n)$ algorithm instead of $O(n^2)$.

✓ Summary:

These slides are the recursive definition of FFT:

- Split coefficients into even and odd parts.
- Recursively evaluate them at squared roots of unity.
- Combine with butterfly operations.
- Repeat until base case.

Would you like me to now **do a worked-out numeric example** (say with $n = 4$) so you can see these butterfly steps with actual numbers?



yes do

Awesome—here's a fully worked **4-point FFT** example, step by step, so you can see the recursion and butterfly combine clearly.

Example setup

Let

$$P(x) = p_0 + p_1x + p_2x^2 + p_3x^3, \quad (p_0, p_1, p_2, p_3) = (1, 2, 3, 4).$$

We'll evaluate P at the **4th roots of unity**:

$$\omega = e^{2\pi i/4} = i, \quad \{\omega^0, \omega^1, \omega^2, \omega^3\} = \{1, i, -1, -i\}.$$

1) Split into even/odd parts (degree halves)

Write

$$P(x) = P_e(x^2) + x P_o(x^2),$$

where

$P_e(t) = p_0 + p_2 t = 1 + 3t$ (even coefficients) and

$P_o(t) = p_1 + p_3 t = 2 + 4t$ (odd coefficients).

We will need P_e and P_o at the **squared nodes** $\{\omega^{2j}\}$ which are the **2nd roots of unity**:

$$\{\omega^0, \omega^2\} = \{1, -1\}.$$

2) Recursive FFT calls on size 2 (base butterflies)

FFT on P_e at $\{1, -1\}$:

- $y_e[0] = P_e(1) = 1 + 3(1) = 4$
- $y_e[1] = P_e(-1) = 1 + 3(-1) = -2$

FFT on P_o at $\{1, -1\}$:

- $y_o[0] = P_o(1) = 2 + 4(1) = 6$
- $y_o[1] = P_o(-1) = 2 + 4(-1) = -2$

(These are just size-2 DFTs: $[a, b] \mapsto [a + b, a - b]$.)

3) Combine (butterfly) to get size-4 results

For $j = 0, 1$ with twiddle ω^j :

- $j = 0$ ($\omega^0 = 1$)
 - $P(\omega^0) = P(1) = y_e[0] + 1 \cdot y_o[0] = 4 + 6 = 10$
 - $P(\omega^{0+2}) = P(\omega^2) = P(-1) = y_e[0] - 1 \cdot y_o[0] = 4 - 6 = -2$
- $j = 1$ ($\omega^1 = i$)
 - $P(\omega^1) = P(i) = y_e[1] + i \cdot y_o[1] = (-2) + i(-2) = -2 - 2i$
 - $P(\omega^{1+2}) = P(\omega^3) = P(-i) = y_e[1] - i \cdot y_o[1] = (-2) - i(-2) = -2 + 2i$

4) Final FFT output (values at roots of unity)

$$[P(1), P(i), P(-1), P(-i)] = [10, -2 - 2i, -2, -2 + 2i]$$

5) Quick cross-check (direct evaluation)

- $P(1) = 1 + 2 + 3 + 4 = 10$ ✓
- $P(-1) = 1 - 2 + 3 - 4 = -2$ ✓
- $P(i) = 1 + 2i + 3(-1) + 4(-i) = -2 - 2i$ ✓
- $P(-i) = 1 - 2i + 3(-1) + 4(i) = -2 + 2i$ ✓

What you just saw

- **Split** into even/odd $\rightarrow$ two size- $n/2$ problems.
- **Evaluate** them at squared nodes (roots of unity stay roots of unity).
- **Combine** with twiddles ω^j via two equations (the butterfly).
- Cost per level is $O(n)$, and there are $\log_2 n$ levels $\rightarrow O(n \log n)$.

If you want, I can do the same walkthrough for a **real-valued input** that's symmetric (like a signal) and show how the complex parts pair up, or give you a tiny **C++/Python FFT** that prints these exact steps.



```
def FFT(P):
    # P = [p0, p1, ..., pn-1] coeff representation
    n = len(P) # n is a power of 2
    if n == 1:
        return P
     $\omega = e^{\frac{2\pi i}{n}}$ 
    Pe, Po = [p0, p2, ..., pn-2], [p1, p3, ..., pn-1]
    ye, yo = FFT(Pe), FFT(Po)
    y = [0] * n
    for j in range(n/2):
        y[j] = ye[j] +  $\omega^j$  yo[j]
        y[j + n/2] = ye[j] -  $\omega^j$  yo[j]
    return y
```

explain this pseudo code of fft

Perfect 🙌 — this is the **FFT algorithm in pseudocode** (Python-like). Let's go through it step by step and connect it to what we've learned.

◆ Code Recap

python

```
def FFT(P):
    # P = [p0, p1, ..., p(n-1)] coefficient representation
    n = len(P) # n is a power of 2

    if n == 1:
        return P # base case: constant polynomial

     $\omega = e^{(2\pi i / n)}$  # primitive nth root of unity

    # Split coefficients into even and odd parts
    Pe = [p0, p2, ..., p(n-2)]
    Po = [p1, p3, ..., p(n-1)]

    # Recursive FFT calls
    ye = FFT(Pe)
    yo = FFT(Po)

    # Allocate result array
    y = [0] * n

    # Combine step (butterfly)
    for j in range(n/2):
        y[j] = ye[j] +  $\omega^j$  * yo[j]
        y[j+n/2] = ye[j] -  $\omega^j$  * yo[j]

    return y
```

◆ Step-by-step Explanation

1. Input

- P is the coefficient list $[p_0, p_1, \dots, p_{n-1}]$.
- Length n is assumed to be a power of 2 (important for recursive halving).

2. Base Case

```
python
if n == 1:
    return P
```

- A degree-0 polynomial (just a constant) evaluates to itself.
- This is the recursion stop.

3. Root of Unity

```
python
ω = e^(2πi / n)
```

- This is the primitive n -th root of unity.
- All evaluation points are powers of this ω .

4. Split Coefficients

```
python
Pe = [p0, p2, ..., p(n-2)]
Po = [p1, p3, ..., p(n-1)]
```

- Separate into **even-index coefficients** and **odd-index coefficients**.
- This is the divide step:

$$P(x) = P_e(x^2) + xP_o(x^2)$$

5. Recursive FFT

python

```
ye = FFT(Pe)
yo = FFT(Po)
```

- Recursively evaluate P_e and P_o at the **squared roots of unity** (ω^2 , which are $(n/2)$ -th roots).
- Results:
 - $ye[j] = P_e(\omega^{2j})$
 - $yo[j] = P_o(\omega^{2j})$

6. Combine (Butterfly Step)

python

```
for j in range(n/2):
    y[j]      = ye[j] + omega^j * yo[j]
    y[j+n/2]  = ye[j] - omega^j * yo[j]
```

- For each j , compute:

$$P(\omega^j) = P_e(\omega^{2j}) + \omega^j P_o(\omega^{2j})$$

$$P(\omega^{j+n/2}) = P_e(\omega^{2j}) - \omega^j P_o(\omega^{2j})$$

- This is the **butterfly operation**: each pair of recursive results gets combined into two new outputs.

7. Return Result

python

```
return y
```

- y now contains $([P(1), P(\omega), P(\omega^2), \dots, P(\omega^{n-1})])$ — the DFT of the input.

Big Picture

- **Divide:** Split polynomial into even and odd parts.
- **Conquer:** Recursively compute FFT on size $n/2$.
- **Combine:** Use butterfly operations to merge into size n .
- Complexity: $O(n \log n)$.

✓ So this pseudocode is literally the **recursive FFT algorithm** that computes the DFT of n coefficients by exploiting roots of unity symmetry and recursion.

Do you want me to also **show the iterative version** (in-place, bit-reversal order) that's used in practice for large FFTs, or should we stick with the recursive version for now?



summarize every step in details up to this point

Perfect 🧡 — let's carefully summarize everything we've built up step by step. This will give you a **clear big picture** of FFT from the ground up.

◆ Step 1: Polynomial Representations

- A degree- d polynomial can be represented in two ways:

1. Coefficient form:

$$P(x) = p_0 + p_1x + \dots + p_dx^d$$

2. Point-value form:

$$\{(x_0, P(x_0)), (x_1, P(x_1)), \dots, (x_d, P(x_d))\}$$

- Both are equivalent: $d + 1$ points uniquely define a degree- d polynomial.

◆ Step 2: Why Point Representation?

- Polynomial multiplication in coefficient form is slow: $O(n^2)$.
- In point-value form, multiplication is just **pointwise multiplication** (fast: $O(n)$).
- So: **Convert coefficients** → **points, multiply, convert back**.

◆ Step 3: The Problem with Naive Evaluation

- To get point representation, we must evaluate the polynomial at many points.
- Naively, evaluating n points costs $O(n^2)$.
- This defeats the purpose — we want something faster.

◆ Step 4: Even/Odd Splitting Trick

- Write

$$P(x) = P_e(x^2) + xP_o(x^2)$$

- P_e : polynomial with even coefficients.
- P_o : polynomial with odd coefficients.
- This reduces evaluating a size- n polynomial into **two polynomials of size $n/2$** .
- **Divide and conquer** structure begins here.

◆ Step 5: The Pairing Symmetry

- If we evaluate at $\pm x$:

$$P(x) = P_e(x^2) + xP_o(x^2), \quad P(-x) = P_e(x^2) - xP_o(x^2)$$

- Both need the **same sub-results** $P_e(x^2), P_o(x^2)$.
- So there's lots of overlap → reuse work.
-  Good symmetry at the first level.

◆ Step 6: Why Arbitrary Points Fail

- With arbitrary choices (like $\{1, 2, 3, 4, \dots\}$), squaring destroys $\pm$ symmetry: $\{1, 2, 3, 4\} \rightarrow \{1, 4, 9, 16\}$.
- Not symmetric $\rightarrow$ recursion breaks.
- We need special points where squaring preserves symmetry.

◆ Step 7: Roots of Unity

- Solutions of $z^n = 1$:

$$\omega^k = e^{2\pi i k/n}, \quad k = 0, \dots, n-1$$

- Geometric interpretation: n equally spaced points on the unit circle.
- Properties:
 1. **$\pm$ Pairing:** $\omega^{j+n/2} = -\omega^j$.
 2. **Closure under squaring:** squaring n -th roots gives $(n/2)$ -th roots.
-  These properties guarantee recursion works.

◆ Step 8: Recursive FFT Structure

- Input: coefficients $[p_0, p_1, \dots, p_{n-1}]$.
- Steps:
 1. Base case: if $n = 1$, return constant.
 2. Split into even/odd coefficient lists.
 3. Recursively compute FFT on them (size $n/2$).
 4. Combine using **butterfly**:

$$P(\omega^j) = y_e[j] + \omega^j y_o[j]$$

$$P(\omega^{j+n/2}) = y_e[j] - \omega^j y_o[j]$$

- This takes $O(n)$ work per level, with $\log n$ levels $\rightarrow$ total $O(n \log n)$.

◆ Step 9: Example (4-point FFT)

- Polynomial: $P(x) = 1 + 2x + 3x^2 + 4x^3$.
- Evaluate at $\{1, i, -1, -i\}$.
- Recursively split into $P_e(t) = 1 + 3t, P_o(t) = 2 + 4t$.
- Compute sub-results at $\{1, -1\}$.
- Combine with butterflies.
- Final FFT output:
 $[10, -2 - 2i, -2, -2 + 2i]$.

◆ Step 10: Pseudocode (Recursive FFT)

python

```
def FFT(P):
    n = len(P)
    if n == 1:
        return P
    ω = exp(2πi/n)
    Pe = [p0, p2, ..., p(n-2)]
    Po = [p1, p3, ..., p(n-1)]
    ye = FFT(Pe)
    yo = FFT(Po)
    y = [0]*n
    for j in range(n//2):
        y[j] = ye[j] + ω^j * yo[j]
        y[j+n//2] = ye[j] - ω^j * yo[j]
    return y
```

- Divide into even/odd parts.
- Recursive FFT on size $n/2$.
- Combine with butterfly.
- Return evaluations at all n -th roots of unity.

✓ Summary in one line:

FFT works because **roots of unity preserve symmetry under squaring**, allowing recursive even/odd splitting and recombination with butterflies, turning polynomial evaluation (DFT) from $O(n^2)$ into $O(n \log n)$.

Do you want me to now also **summarize the big picture with a diagram/flow (Coefficient form → FFT → Multiply → IFFT → Result)** so you see how this ties back

to fast polynomial multiplication?



Interpolation

Alternative Perspective on Evaluation/FFT

$$P(x) = p_0 + p_1x + p_2x^2 + \cdots + p_{n-1}x^{n-1}$$

$$\begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix}}_{\text{Discrete Fourier Transform (DFT) matrix}} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

Discrete Fourier Transform (DFT) matrix

$$x_k = \omega^k \text{ where } \omega = e^{\frac{2\pi i}{n}}$$

Interpolation

Interpolation involves inverting the DFT matrix

$$P(x) = p_0 + p_1x + p_2x^2 + \cdots + p_{n-1}x^{n-1}$$

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

$$x_k = \omega^k \text{ where } \omega = e^{\frac{2\pi i}{n}}$$

Perfect timing 🚀 — these two slides connect FFT back to **linear algebra** and explain **interpolation** (the inverse FFT). Let's unpack them carefully.

◆ Slide 1: Evaluation as Matrix Multiplication

We want to evaluate:

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_{n-1}x^{n-1}$$

At the n -th roots of unity:

$$\{\omega^0, \omega^1, \dots, \omega^{n-1}\}, \quad \omega = e^{2\pi i/n}$$

This can be written in **matrix form**:

$$\begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

✓ This matrix is the **Discrete Fourier Transform (DFT) matrix**.

So the FFT is just a **fast algorithm to multiply this special matrix** by the coefficient vector.

◆ Slide 2: Interpolation (Inverse FFT)

Now suppose instead of coefficients we are given the values $[P(\omega^0), P(\omega^1), \dots, P(\omega^{n-1})]$.

We want to recover coefficients $[p_0, p_1, \dots, p_{n-1}]$.

This means:

$$\begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{n-1} \end{bmatrix} = (\text{DFT matrix})^{-1} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

So **interpolation** is equivalent to applying the **inverse DFT**.

◆ Key Facts about the DFT Matrix

- The DFT matrix is a special **Vandermonde matrix** built from powers of ω .

- It is **unitary** (up to scaling):

$$F^{-1} = \frac{1}{n} \overline{F}^T$$

where $\overline{F}^T$ is the conjugate transpose of F .

- This is why the **inverse FFT** looks almost the same as FFT, except:
 - We use $\omega^{-1} = e^{-2\pi i/n}$ instead of ω .
 - At the end, divide everything by n .

◆ Big Picture Connection

1. **Evaluation (FFT)**: Multiply coefficient vector by the DFT matrix → get values at roots of unity.
2. **Interpolation (Inverse FFT)**: Multiply values by the inverse DFT matrix → recover coefficients.

This symmetry is what makes FFT so powerful:

- Forward FFT: coefficients → values.
- Multiply values pointwise (cheap).
- Inverse FFT: values → coefficients.

✓ So these slides show the **linear algebra viewpoint**: FFT is just a fast algorithm for multiplying by the DFT matrix (and its inverse).

Do you want me to next **show the entire flow of polynomial multiplication** (Coefficient form → FFT → multiply in point form → IFFT → result), step by step with an example?



$$x_k = \omega^k \text{ where } \omega = e^{\frac{2\pi i}{n}}$$

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

$$\Downarrow$$

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \dots & \omega^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

The inverse matrix and original matrix look quite similar!

Every ω in original matrix is now $\frac{1}{n}\omega^{-1}$

Great — this slide is showing the **Inverse FFT (IFFT)** more explicitly, in terms of the DFT matrix. Let's carefully break it down.

◆ Top: Forward Transform (Evaluation)

We know the forward FFT (DFT) is:

$$\begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ P(\omega^2) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}}_{\text{DFT matrix } F} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

◆ Middle: Inverse Transform

To get back coefficients, we invert the matrix:

$$\begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{n-1} \end{bmatrix} = F^{-1} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

◆ Bottom: Structure of the Inverse Matrix

The magic is that F^{-1} looks almost exactly like F , except:

1. Replace ω with ω^{-1} .
 - Forward DFT uses powers of ω .
 - Inverse DFT uses powers of ω^{-1} .
2. Scale everything by $1/n$.
 - Because the DFT matrix is *unitary* up to a factor of n .

So:

$$F^{-1} = \frac{1}{n} \overline{F}^T$$

Concretely:

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \dots & \omega^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

Key Takeaways

- **Forward FFT:** use $\omega = e^{2\pi i/n}$.
- **Inverse FFT:** use $\omega^{-1} = e^{-2\pi i/n}$, then divide results by n .
- Both look nearly identical → that's why in code, the same FFT routine can often compute both (just flip the sign in the exponent and rescale).

Summary in plain words:

- FFT converts coefficients → values (at roots of unity).
- Inverse FFT converts values → coefficients.
- The two operations are almost the same; only difference: flip the direction of rotation (use ω^{-1}) and divide by n .

Would you like me to now **show the full polynomial multiplication pipeline** (Coeff → FFT → multiply pointwise → IFFT → result) with an example to tie everything together?



$\text{IFFT}(\langle \text{values} \rangle) \Leftrightarrow \text{FFT}(\langle \text{values} \rangle)$ with $\omega = \frac{1}{n} e^{-\frac{2\pi i}{n}}$

<pre>def FFT(P): # P = [p0, p1, ..., pn-1] coeff rep n = len(P) # n is a power of 2 if n == 1: return P $\omega = e^{\frac{2\pi i}{n}}$ Pe, Po = P[:2], P[1::2] ye, yo = FFT(Pe), FFT(Po) y = [0] * n for j in range(n/2): y[j] = ye[j] + ω^j yo[j] y[j+n/2] = ye[j] - ω^j yo[j] return y</pre>	<pre>def IFFT(P): # P = [P(ω^0), P(ω^1), ..., P(ω^{n-1})] value rep n = len(P) # n is a power of 2 if n == 1: return P $\omega = (1/n) * e^{-\frac{2\pi i}{n}}$ Pe, Po = P[:2], P[1::2] ye, yo = IFFT(Pe), IFFT(Po) y = [0] * n for j in range(n/2): y[j] = ye[j] + ω^j yo[j] y[j+n/2] = ye[j] - ω^j yo[j] return y</pre>
--	--

Perfect — this slide puts **FFT and IFFT side by side** and shows how similar they are. Let's carefully explain it.

◆ Left: FFT (Forward Transform)

python

```
def FFT(P):
    n = len(P)
    if n == 1:
        return P
     $\omega = e^{(2\pi i/n)}$ 
    Pe, Po = P[:2], P[1::2] # even, odd coefficients
    ye, yo = FFT(Pe), FFT(Po) # recursive calls
    y = [0] * n
    for j in range(n//2):
        y[j] = ye[j] +  $\omega^j$  * yo[j]
        y[j+n//2] = ye[j] -  $\omega^j$  * yo[j]
    return y
```

- Input: coefficient representation $[p_0, p_1, \dots, p_{(n-1)}]$.
- Output: polynomial values at roots of unity $[P(\omega^0), P(\omega^1), \dots, P(\omega^{(n-1)})]$.
- Uses primitive root of unity:
 $\omega = e^{2\pi i/n}$.

◆ Right: IFFT (Inverse Transform)

python

```
def IFFT(P):
    n = len(P)
    if n == 1:
        return P
    ω = (1/n) * e^(-2πi/n)
    Pe, Po = P[::2], P[1::2]
    ye, yo = IFFT(Pe), IFFT(Po)
    y = [0] * n
    for j in range(n//2):
        y[j] = ye[j] + ω^j * yo[j]
        y[j+n//2] = ye[j] - ω^j * yo[j]
    return y
```

- Input: value representation $[P(\omega^0), \dots, P(\omega^{(n-1)})]$.
- Output: coefficient representation $[p_0, p_1, \dots, p_{(n-1)}]$.
- Uses the **inverse root of unity**:
 $\omega = e^{-2\pi i/n}$.
- Includes a factor of $1/n$ for normalization.

◆ Key Observations

1. **Structure is identical:** Both recursively split into even and odd parts, recurse, then combine with butterflies.
2. **Difference in ω :**
 - FFT uses $e^{+2\pi i/n}$.
 - IFFT uses $e^{-2\pi i/n}$.
3. **Scaling by $1/n$:** Needed in IFFT to normalize, since DFT matrix is not orthonormal but unitary up to factor n .

Summary

- **FFT:** Coefficients $\rightarrow$ Values (evaluation).
- **IFFT:** Values $\rightarrow$ Coefficients (interpolation).
- They are essentially the *same algorithm* with just two tweaks:
 - Flip sign in exponent (+ vs. -).

- Divide by n in the inverse.

That's why in most practical codebases, FFT and IFFT are implemented using the *same function* with a flag for "inverse".

✅ Now you have the **full forward + inverse pipeline**.

Would you like me to **show the complete polynomial multiplication process with FFT + IFFT (step by step with numbers)** so you see how both transforms work together in action?